

CatchFood y Menu Principal

Guia de estudio completa con explicacion y codigo actual

Objetivo: Entender como funciona el juego CatchFood y el menu principal de PlayLand desde la interfaz, la navegacion, la logica, el renderizado y la persistencia local.

Esta guia esta escrita para estudiar y defender el codigo. Incluye explicaciones por partes y, al final, el codigo completo de los archivos relacionados.

Tema	Que debes poder explicar
Menu principal	Como se arma la lista de juegos, como funcionan los botones y como se invoca la navegacion.
CatchFood menu	Como se carga el fondo, como se ubican botones transparentes y como se entra al juego o al historico.
Juego	Como se actualiza cada frame, como se dibujan sprites, como se mueve el jugador y como se detecta derrota.
ViewModel	Como separa UI y logica, sincroniza UiState y guarda puntajes al finalizar.
Room	Como Entity, DAO, Database y Repository guardan y consultan el TOP 10.

1. Resumen general

PlayLand tiene un menu principal que presenta cuatro juegos. Cada boton llama una funcion de navegacion recibida por parametro. Eso permite que el menu sea una pantalla visual y no tenga que saber como esta construido el NavHost principal.

CatchFood es un modulo con navegacion interna: primero muestra su menu, desde ahi entra al juego o al historico, y dentro del juego el usuario controla un personaje para atrapar comida y evitar veneno.

2. Menu principal nuevo

MenuScreen recibe cuatro callbacks: uno para Snake, uno para Tic Tac Toe, uno para CatchFood y uno para Flappy Bird. Un callback es una funcion que la pantalla ejecuta cuando ocurre un evento, por ejemplo tocar un boton.

Dentro del Composable se crea una fuente con Fredoka y una lista games. Cada MenuGame contiene titulo, subtítulo, inicial, color y accion. Luego GameMenuButton pinta cada boton usando esa informacion.

3. Menu interno de CatchFood

CatchFoodScreen usa LocalContext para acceder a assets, carga background_records.jpg y lo muestra como fondo. Los botones son transparentes porque la imagen ya tiene el diseno visual; el boton solo define el area tactil.

El boton de jugar ejecuta onGoToGame, el historico ejecuta onGoToLeaderboard y volver ejecuta onBack.

4. Navegacion interna

CatchFoodNavHost crea un NavController interno con rememberNavController. El destino inicial es CatchFoodMenu. Desde ahi navega a CatchFoodGameScreen o CatchFoodLeaderboardScreen.

Los destinos estan declarados como object con @Serializable para usar rutas tipadas en Navigation Compose.

5. Pantalla de juego

CatchFoodGame obtiene contexto, densidad de pantalla, ViewModel y UiState. Carga los sprites con CatchFoodBitmaps.load dentro de remember para no decodificar imagenes en cada recomposicion.

El loop usa LaunchedEffect y withFrameNanos. En cada frame calcula deltaSeconds, que representa el tiempo real entre frames, y llama updateGame cuando la partida no esta pausada ni terminada.

La escena usa Canvas para dibujar comida, veneno y jugador en una sola superficie animada.

6. Logica del juego

CatchFoodGameLogic guarda posicion, puntaje, golpes de veneno, comida perdida, velocidad, estado visual y lista de objetos. Las constantes privadas tienen nombres descriptivos en espanol como fpsObjetivo, velocidadInicial, cantidadObjetos y anchoHitboxJugadorDp.

updateGame calcula escalaDelFrame con deltaSeconds, aumenta speed, mueve cada objeto y calcula colisiones con hitboxes rectangulares.

Si toca comida aumenta score. Si toca veneno aumenta poisonHits. Con tres venenos o cinco comidas perdidas, isGameOver pasa a true.

resetObject vuelve a colocar el objeto arriba, detras de la cola de objetos, con una nueva posicion horizontal aleatoria.

Fragmento clave

```
val escalaDelFrame = (deltaSeconds * fpsObjetivo).coerceIn(0f, escalaMaximaFrame)
speed += aumentoVelocidadPorFrame * escalaDelFrame

val tocaJugador = viewportHeightDp > 0f &&
    izquierdaObjeto < izquierdaJugador + anchoHitboxJugadorDp &&
    izquierdaObjeto + anchoHitboxObjetoDp > izquierdaJugador
```

7. ViewModel y UiState

CatchFoodViewModel coordina CatchFoodGameLogic y CatchFoodUiState. La UI observa uiState y Compose recompone cuando cambia score, poisonHits, missedFood, playerState, isPaused o isGameOver.

Cuando termina la partida, el ViewModel guarda el puntaje una sola vez usando scoreSavedForCurrentGame y viewModelScope.launch.

8. Historico y Room

Room guarda los puntajes localmente sobre SQLite. CatchFoodScoreEntity define la tabla, CatchFoodScoreDao declara insercion y consulta TOP 10, CatchFoodDatabase crea la base y CatchFoodScoreRepository oculta los detalles de Room.

CatchFoodLeaderboard usa produceState para ejecutar la consulta suspendida y convertir el resultado en estado observable. LazyColumn muestra el ranking.

9. Recorrido completo de una partida

1. El usuario entra desde el menu principal a Catch Food.
2. CatchFoodNavHost muestra CatchFoodScreen como destino inicial.
3. El usuario toca jugar y se navega a CatchFoodGameScreen.
4. CatchFoodGame obtiene el ViewModel y carga sprites.
5. LaunchedEffect inicia el loop sincronizado con frames.
6. updateGame mueve objetos, aumenta velocidad y revisa colisiones.
7. El usuario arrastra y movePlayer limita la posicion horizontal con coerceIn.
8. Si atrapa comida aumenta score; si toca veneno aumenta poisonHits.
9. Si pierde cinco comidas o toca tres venenos, isGameOver se activa.
10. El ViewModel guarda el puntaje final una sola vez con Room.

11. Al entrar al historico, Room devuelve los diez mejores puntajes.

10. Preguntas comunes de defensa

Pregunta	Respuesta corta
Que es una lambda?	Una funcion anonima que se pasa como dato. Se usa en onClick y callbacks de navegacion.
Que es UiState?	Una data class con los valores visibles de la pantalla.
Por que ViewModel?	Para separar estado y logica de la UI, y usar viewModelScope para trabajo asincrono.
Por que Canvas?	Porque la escena animada se dibuja mejor en una sola superficie.
Como se detecta colision?	Comparando rectangulos de hitbox con condiciones AABB.
Que es Room?	Una libreria Android sobre SQLite para persistencia local.

11.Codigo completo

A continuacion esta el codigo completo de los archivos relacionados con el menu nuevo y CatchFood. Estudialo por responsabilidad: primero UI, luego navegacion, luego logica, luego estado y datos.

app/src/main/java/com/example/playland2/feature/menu/MenuScreen.kt

```
package com.example.playland2.feature.menu

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.ElevatedButton
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.Font
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.playland2.R

@Composable
fun MenuScreen(
    onNavigateToSnake: () -> Unit,
    onNavigateToTicTacToe: () -> Unit,
    onNavigateToCatchFood: () -> Unit,
    onNavigateToFlappyBird: () -> Unit
) {
    val gameFont = remember { FontFamily(Font(R.font.fredoka_bold)) }
    val games = listOf(
        MenuGame("Snake", "Desliza y crece", "S", Color(0xFF4F8D3A), onNavigateToSnake),
        MenuGame("Flappy Bird", "Vuela sin caer", "F", Color(0xFF3F7CAC),
            onNavigateToFlappyBird),
        MenuGame("Catch Food", "Atrapa comida", "C", Color(0xFFE96E9D),
            onNavigateToCatchFood),
        MenuGame("Tic Tac Toe", "Tres en raya", "T", Color(0xFFD95B55),
            onNavigateToTicTacToe)
    )

    Box(
        modifier = Modifier
            .fillMaxSize()
            .background(Color(0xFFFFCF5E4))
            .padding(horizontal = 28.dp, vertical = 44.dp)
    ) {
        Column(
            modifier = Modifier.fillMaxSize(),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(

```

continuacion

```

        text = "PLAYLAND",
        fontFamily = gameFont,
        fontSize = 46.sp,
        color = Color(0xFFE96E9D)
    )
    Text(
        text = "Menu de juegos",
        fontFamily = gameFont,
        fontSize = 22.sp,
        color = Color(0xFF6B3F2D)
    )

    Spacer(modifier = Modifier.height(34.dp))

    Column(
        modifier = Modifier.fillMaxWidth(),
        verticalArrangement = Arrangement.spacedBy(14.dp)
    ) {
        games.forEach { game ->
            GameMenuButton(game = game, gameFont = gameFont)
        }
    }

    Spacer(modifier = Modifier.weight(1f))

    Text(
        text = "Elige tu partida",
        fontFamily = gameFont,
        fontSize = 18.sp,
        color = Color(0xFF6B3F2D)
    )
}
}

@Composable
private fun GameMenuButton(
    game: MenuGame,
    gameFont: FontFamily
) {
    ElevatedButton(
        onClick = game.onClick,
        modifier = Modifier
            .fillMaxWidth()
            .height(78.dp),
        colors = ButtonDefaults.elevatedButtonColors(
            containerColor = Color.White,
            contentColor = Color(0xFF6B3F2D)
        ),
        elevation = ButtonDefaults.elevatedButtonElevation(defaultElevation = 3.dp),
        shape = RoundedCornerShape(18.dp)
    ) {
        Row(
            modifier = Modifier.fillMaxWidth(),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Box(
                modifier = Modifier

```

continuacion

```

        .width(52.dp)
        .height(46.dp)
        .background(game.color, RoundedCornerShape(14.dp)),
        contentAlignment = Alignment.Center
    ) {
        Text(
            text = game.initial,
            fontFamily = gameFont,
            fontSize = 24.sp,
            color = Color.White
        )
    }

    Column(
        modifier = Modifier
            .weight(1f)
            .padding(start = 18.dp),
        verticalArrangement = Arrangement.Center
    ) {
        Text(
            text = game.title,
            fontFamily = gameFont,
            fontWeight = FontWeight.Bold,
            fontSize = 22.sp,
            color = Color(0xFF6B3F2D)
        )
        Text(
            text = game.subtitle,
            fontFamily = gameFont,
            fontSize = 14.sp,
            color = Color(0xFF8B6B56)
        )
    }

    Text(
        text = ">",
        fontFamily = gameFont,
        fontSize = 28.sp,
        color = game.color
    )
}

private data class MenuGame(
    val title: String,
    val subtitle: String,
    val initial: String,
    val color: Color,
    val onClick: () -> Unit
)

```

app/src/main/java/com/example/playland2/feature/catchfood/ CatchFoodScreen.kt

```
package com.example.playland2.feature.catchfood

import android.graphics.BitmapFactory
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.offset
import androidx.compose.foundation.layout.size
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.runtime.Composable
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.asImageBitmap
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.unit.dp

@Composable
fun CatchFoodScreen(
    onBack: () -> Unit,
    onGoToGame: () -> Unit,
    onGoToLeaderboard: () -> Unit
) {

    val context = LocalContext.current

    val fondoCompleto = remember {

        try {

            context.assets.open("games/catchfood/sprites/background_records.jpg")
                .use { inputStream ->

                    BitmapFactory.decodeStream(inputStream).asImageBitmap()

                }

        } catch (e: Exception) {

            null

        }
    }

    Box(
        modifier = Modifier
            .fillMaxSize()
            .background(Color(0xFFFFCF5E4))
    ) {

        fondoCompleto?.let { bitmap ->

            Image(
                bitmap = bitmap,
                contentDescription = "Fondo de CatchFood",
                modifier = Modifier.fillMaxSize(),
            )
        }
    }
}
```

continuacion

```

        contentScale = ContentScale.FillBounds,
        alignment = Alignment.Center
    )
}

Button(
    onClick = { onGoToGame()},

    modifier = Modifier
        .align(Alignment.Center)
        .offset(
            x = 0.dp,
            y = 25.dp
        )
        .size(
            width = 260.dp,
            height = 70.dp
        ),

    colors = ButtonDefaults.buttonColors(
        containerColor = Color.Transparent
    )
) {
}

Button(
    onClick = onGoToLeaderboard,

    modifier = Modifier
        .align(Alignment.Center)
        .offset(
            x = 0.dp,
            y = 140.dp
        )
        .size(
            width = 260.dp,
            height = 70.dp
        ),

    colors = ButtonDefaults.buttonColors(
        containerColor = Color.Transparent
    )
) {
}

Button(
    onClick = { onBack() },

    modifier = Modifier
        .align(Alignment.Center)
        .offset(
            x = 0.dp,
            y = 240.dp
        )
        .size(
            width = 260.dp,
            height = 70.dp
        ),

```

continuacion

```
        ),  
        colors = ButtonDefaults.buttonColors(  
            containerColor = Color.Transparent  
        )  
    }  
}
```

app/src/main/java/com/example/playland2/feature/catchfood/ ui/navigation/CatchFoodDestinations.kt

```
package com.example.playland2.feature.catchfood.ui.navigation

import kotlinx.serialization.Serializable

@Serializable
object CatchFoodMenu

@Serializable
object CatchFoodGameScreen

@Serializable
object CatchFoodLeaderboardScreen
```

app/src/main/java/com/example/playland2/feature/catchfood/ui/navigation/CatchFoodNavHost.kt

```
package com.example.playland2.feature.catchfood.ui.navigation

import androidx.compose.runtime.Composable
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.rememberNavController
import com.example.playland2.feature.catchfood.CatchFoodScreen
import com.example.playland2.feature.catchfood.ui.screen.CatchFoodGame
import com.example.playland2.feature.catchfood.ui.screen.CatchFoodLeaderboard

@Composable
fun CatchFoodNavHost(
    onBack: () -> Unit
) {
    val navController = rememberNavController()

    NavHost(
        navController = navController,
        startDestination = CatchFoodMenu
    ) {
        // menu principal el del historico
        composable<CatchFoodMenu> {
            CatchFoodScreen(
                onBack = {
                    onBack()
                },
                onGoToGame = {
                    navController.navigate(
                        CatchFoodGameScreen
                    )
                },
                onGoToLeaderboard = {
                    navController.navigate(CatchFoodLeaderboardScreen)
                }
            )
        }

        // jueguito
        composable<CatchFoodGameScreen> {
            CatchFoodGame(
                onBack = {
                    navController.popBackStack()
                }
            )
        }

        composable<CatchFoodLeaderboardScreen> {
```

continuacion

```
        CatchFoodLeaderboard(  
            onBack = { navController.popBackStack() }  
        )  
    }  
}
```

app/src/main/java/com/example/playland2/feature/catchfood/ui/screen/CatchFoodGame.kt

```
package com.example.playland2.feature.catchfood.ui.screen

import androidx.compose.foundation.Canvas
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.gestures.detectDragGestures
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.offset
import androidx.compose.foundation.layout.size
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.remember
import androidx.compose.runtime.withFrameNanos
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.ImageBitmap
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.platform.LocalDensity
import androidx.compose.ui.text.font.Font
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.unit.IntOffset
import androidx.compose.ui.unit.IntSize
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewModel.compose.viewModel
import com.example.playland2.R

@Composable
fun CatchFoodGame(onBack: () -> Unit) {
    val context = LocalContext.current
    val density = LocalDensity.current.density
    val gameViewModel: CatchFoodViewModel = viewModel()
    val gameLogic = gameViewModel.gameLogic
    val uiState = gameViewModel.uiState

    val bitmaps = remember(context, density) {
        CatchFoodBitmaps.load(context, density)
    }

    LaunchedEffect(gameViewModel) {
        var previousFrameNanos = 0L
        while (true) {
            withFrameNanos { frameNanos ->
                if (!gameViewModel.uiState.isPaused && !gameViewModel.uiState.isGameOver) {
                    if (previousFrameNanos != 0L) {
                        val deltaSeconds = (frameNanos - previousFrameNanos) /
                            1_000_000_000f
                        gameViewModel.updateGame(deltaSeconds)
                    }
                    previousFrameNanos = frameNanos
                } else {

```

continuacion

```

        previousFrameNanos = 0L
    }
}

val gameFont = remember { FontFamily(Font(R.font.fredoka_bold)) }

Box(
    modifier = Modifier
        .fillMaxSize()
        .background(Color(0xFFFCF5E4))
) {
    GameScene(
        gameLogic = gameLogic,
        onMovePlayer = gameViewModel::movePlayer,
        onViewportHeightChanged = gameViewModel::setViewportHeight,
        foodBitmap = bitmaps.food,
        poisonBitmap = bitmaps.poison,
        playerBitmap = when (uiState.playerState) {
            "hurt" -> bitmaps.hurtPlayer
            "dead" -> bitmaps.deadPlayer
            else -> bitmaps.player
        }
    )

    bitmaps.title?.let {
        Image(
            bitmap = it,
            contentDescription = null,
            modifier = Modifier
                .align(Alignment.TopCenter)
                .offset(y = (-10).dp)
                .size(width = 300.dp, height = 180.dp),
            contentScale = ContentScale.Fit
        )
    }

    Column(
        modifier = Modifier
            .align(Alignment.TopStart)
            .offset(x = 20.dp, y = 190.dp)
    ) {
        Text("Comida: ${uiState.score}", fontFamily = gameFont, color = Color.Black)
        Text("Veneno: ${uiState.poisonHits}/3", fontFamily = gameFont, color =
            Color.Red)
        Text("Perdidas: ${uiState.missedFood}/5", fontFamily = gameFont, color =
            Color.DarkGray)
    }

    ImageButton(
        bitmap = bitmaps.pause,
        onClick = gameViewModel::pause,
        modifier = Modifier
            .align(Alignment.TopEnd)
            .offset(y = 160.dp)
            .size(90.dp)
    )
}

```

continuacion

```

if (uiState.isGameOver) {
    Box(Modifier.fillMaxSize().background(Color(0xFFFCF5E4)))

    bitmaps.gameOver?.let { gameOverBitmap ->
        Box(
            modifier = Modifier
                .align(Alignment.Center)
                .offset(y = (-60).dp)
                .size(320.dp)
        ) {
            Image(
                bitmap = gameOverBitmap,
                contentDescription = null,
                modifier = Modifier.fillMaxSize(),
                contentScale = ContentScale.Fit
            )
            TransparentButton(
                onClick = gameViewModel::restart,
                modifier = Modifier
                    .offset(x = 12.dp, y = 190.dp)
                    .size(width = 142.dp, height = 45.dp)
            )
            TransparentButton(
                onClick = onBack,
                modifier = Modifier
                    .offset(x = 167.dp, y = 190.dp)
                    .size(width = 141.dp, height = 45.dp)
            )
        }
    }
}

if (uiState.isPaused) {
    Box(Modifier.fillMaxSize().background(Color.Black.copy(alpha = 0.5f)))

    ImageButton(
        bitmap = bitmaps.continueButton,
        onClick = gameViewModel::resume,
        modifier = Modifier
            .align(Alignment.Center)
            .offset(y = (-90).dp)
            .size(width = 340.dp, height = 120.dp)
    )
    ImageButton(
        bitmap = bitmaps.restart,
        onClick = gameViewModel::restart,
        modifier = Modifier
            .align(Alignment.Center)
            .offset(y = 10.dp)
            .size(width = 390.dp, height = 150.dp)
    )
    ImageButton(
        bitmap = bitmaps.back,
        onClick = onBack,
        modifier = Modifier
            .align(Alignment.Center)
            .offset(y = 110.dp)
    )
}

```

continuacion

```

        .size(width = 340.dp, height = 120.dp)
    )
}
}
}

@Composable
private fun GameScene(
    gameLogic: CatchFoodGameLogic,
    onMovePlayer: (Float) -> Unit,
    onViewportHeightChanged: (Float) -> Unit,
    foodBitmap: ImageBitmap?,
    poisonBitmap: ImageBitmap?,
    playerBitmap: ImageBitmap?
) {
    val density = LocalDensity.current.density

    Canvas(
        modifier = Modifier
            .fillMaxSize()
            .pointerInput(gameLogic) {
                detectDragGestures { _, dragAmount ->
                    onMovePlayer(dragAmount.x)
                }
            }
    ) {
        gameLogic.frameVersion
        onViewportHeightChanged(size.height / density)

        val objectSize = (90f * density).toInt()
        gameLogic.objects.forEach { obj ->
            val bitmap = if (obj.isPoison) poisonBitmap else foodBitmap
            bitmap?.let {
                drawImage(
                    image = it,
                    dstOffset = IntOffset(
                        x = ((obj.x / 3f) * density).toInt(),
                        y = (((obj.y / 3f) + 220f) * density).toInt()
                    ),
                    dstSize = IntSize(objectSize, objectSize)
                )
            }
        }

        playerBitmap?.let {
            val playerSize = (170f * density).toInt()
            drawImage(
                image = it,
                dstOffset = IntOffset(
                    x = ((gameLogic.playerX / 3f) * density).toInt(),
                    y = size.height.toInt() - playerSize - (40f * density).toInt()
                ),
                dstSize = IntSize(playerSize, playerSize)
            )
        }
    }
}
}

```

continuacion

```

@Composable
private fun ImageButton(
    bitmap: ImageBitmap?,
    onClick: () -> Unit,
    modifier: Modifier
) {
    Box(modifier) {
        bitmap?.let {
            Image(
                bitmap = it,
                contentDescription = null,
                modifier = Modifier.fillMaxSize(),
                contentScale = ContentScale.Fit
            )
        }
        Button(
            onClick = onClick,
            modifier = Modifier.fillMaxSize(),
            colors = ButtonDefaults.buttonColors(containerColor = Color.Transparent)
        ) {}
    }
}

@Composable
private fun TransparentButton(
    onClick: () -> Unit,
    modifier: Modifier
) {
    Button(
        onClick = onClick,
        modifier = modifier,
        colors = ButtonDefaults.buttonColors(containerColor = Color.Transparent),
        contentPadding = androidx.compose.foundation.layout.PaddingValues(0.dp)
    ) {}
}

```

app/src/main/java/com/example/playland2/feature/catchfood/ui/screen/CatchFoodGameLogic.kt

```
package com.example.playland2.feature.catchfood.ui.screen

import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableFloatStateOf
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.setValue
import kotlin.random.Random

class CatchFoodGameLogic {

    var playerX by mutableFloatStateOf(300f)
    var score by mutableIntStateOf(0)
    var poisonHits by mutableIntStateOf(0)
    var missedFood by mutableIntStateOf(0)
    var isGameOver by mutableStateOf(false)
    var speed = velocidadInicial
    private set
    var playerState by mutableStateOf("eat")
    val objects = ArrayList<FallingObject>(cantidadObjetos)
    var frameVersion by mutableIntStateOf(0)
    private set
    private var viewportHeightDp = 0f

    init {
        createObjects()
    }

    fun updateGame(deltaSeconds: Float) {

        if (isGameOver) return

        // aqui incremento la velocidad
        val escalaDelFrame = (deltaSeconds * fpsObjetivo).coerceIn(0f, escalaMaximaFrame)
        speed += aumentoVelocidadPorFrame * escalaDelFrame

        objects.forEach { obj ->

            obj.y += speed * escalaDelFrame

            val izquierdaObjeto = obj.x / escalaMundo + margenXHitboxObjetoDp
            val arribaObjeto = obj.y / escalaMundo + distanciaSuperiorObjetoDp
            val izquierdaJugador = playerX / escalaMundo + margenXHitboxJugadorDp
            val arribaJugador = viewportHeightDp - tamanoJugadorDp -
                distanciaInferiorJugadorDp +
                margenSuperiorHitboxJugadorDp

            // collision
            val tocaJugador = viewportHeightDp > 0f &&
                izquierdaObjeto < izquierdaJugador + anchoHitboxJugadorDp &&
                izquierdaObjeto + anchoHitboxObjetoDp > izquierdaJugador &&
                arribaObjeto + margenSuperiorHitboxObjetoDp < arribaJugador +
                altoHitboxJugadorDp &&
                arribaObjeto + margenSuperiorHitboxObjetoDp + altoHitboxObjetoDp >
                arribaJugador

            if (tocaJugador) {
                if (obj.isPoison) {
```

continuacion

```

        poisonHits++
        playerState = "hurt"

        if (poisonHits >= 3) {
            playerState = "dead"
            isGameOver = true
        }
    } else {
        score++
    }

    resetObject(obj)
    return@forEach
}

// objeto perdido
if (viewportHeightDp > 0f && arribaObjeto > viewportHeightDp) {

    if (!obj.isPoison) {

        missedFood++

        if (missedFood >= 5) {

            playerState = "dead"
            isGameOver = true
        }
    }

    resetObject(obj)
}

frameVersion++
}

fun movePlayer(deltaX: Float) {
    playerX = (playerX + deltaX).coerceIn(0f, posicionMaximaJugadorX)
}

fun setViewportHeight(heightDp: Float) {
    viewportHeightDp = heightDp
}

fun restart() {
    objects.clear()
    playerX = 300f
    score = 0
    poisonHits = 0
    missedFood = 0
    speed = velocidadInicial
    playerState = "eat"
    isGameOver = false

    createObjects()
    frameVersion++
}

```

continuacion

```

private fun resetObject(obj: FallingObject) {
    val objetoMasArribaEnCola = objects
        .asSequence()
        .filter { it !== obj }
        .minOfOrNull { it.y }
        ?.posicionInicialAparicionY

    obj.y = minOf(objetoMasArribaEnCola - separacionAparicionMundo,
        posicionInicialAparicionY)
    obj.x = Random.nextInt(80, 850).toFloat()
}

private fun createObjects() {
    val tiposDeObjeto = (List(8) { false } + List(2) { true }).shuffled()
    tiposDeObjeto.forEachIndexed { index, isPoison ->
        objects.add(
            FallingObject(
                x = Random.nextInt(80, 850).toFloat(),
                y = posicionInicialAparicionY - (separacionAparicionMundo * index),
                isPoison = isPoison
            )
        )
    }
}

private companion object {
    const val fpsObjetivo = 60f
    const val escalaMaximaFrame = 3f
    const val aumentoVelocidadPorFrame = 0.001f
    const val velocidadInicial = 8f
    const val posicionMaximaJugadorX = 850f
    const val cantidadObjetos = 10
    const val escalaMundo = 3f
    const val distanciaSuperiorObjetoDp = 220f
    const val tamanoJugadorDp = 170f
    const val distanciaInferiorJugadorDp = 40f
    const val posicionInicialAparicionY = -500f
    const val separacionAparicionMundo = 650f
    const val margenXHitboxJugadorDp = 45f
    const val margenSuperiorHitboxJugadorDp = 50f
    const val anchoHitboxJugadorDp = 80f
    const val altoHitboxJugadorDp = 75f
    const val margenXHitboxObjetoDp = 18f
    const val margenSuperiorHitboxObjetoDp = 12f
    const val anchoHitboxObjetoDp = 54f
    const val altoHitboxObjetoDp = 66f
}
}

```

app/src/main/java/com/example/playland2/feature/catchfood/ui/screen/FallingObject.kt

```
package com.example.playland2.feature.catchfood.ui.screen

data class FallingObject(
    var x: Float,
    var y: Float,
    val isPoison: Boolean
)
```

app/src/main/java/com/example/playland2/feature/catchfood/ui/screen/CatchFoodBitmaps.kt

```
package com.example.playland2.feature.catchfood.ui.screen

import android.content.Context
import android.graphics.BitmapFactory
import androidx.compose.ui.graphics.ImageBitmap
import androidx.compose.ui.graphics.asImageBitmap

data class CatchFoodBitmaps(
    val player: ImageBitmap?,
    val hurtPlayer: ImageBitmap?,
    val deadPlayer: ImageBitmap?,
    val food: ImageBitmap?,
    val poison: ImageBitmap?,
    val title: ImageBitmap?,
    val pause: ImageBitmap?,
    val continueButton: ImageBitmap?,
    val restart: ImageBitmap?,
    val back: ImageBitmap?,
    val gameOver: ImageBitmap?
) {
    companion object {
        fun load(context: Context, density: Float): CatchFoodBitmaps {
            fun load(path: String, widthDp: Int, heightDp: Int) =
                decodeSampledAsset(context, path, widthDp * density, heightDp * density)

            return CatchFoodBitmaps(
                player = load("games/catchfood/sprites/comer.png", 170, 170),
                hurtPlayer = load("games/catchfood/sprites/enfermo.png", 170, 170),
                deadPlayer = load("games/catchfood/sprites/muerto.png", 170, 170),
                food = load("games/catchfood/sprites/zanahoria.png", 90, 90),
                poison = load("games/catchfood/sprites/veneno.png", 90, 90),
                title = load("games/catchfood/sprites/titulo.png", 300, 180),
                pause = load("games/catchfood/sprites/btn_pausa.png", 90, 90),
                continueButton = load("games/catchfood/sprites/btn_continuar.png", 340,
                    120),
                restart = load("games/catchfood/sprites/btn_reiniciar.png", 390, 150),
                back = load("games/catchfood/sprites/btn_volver_menu_principal.png", 340,
                    120),
                gameOver = load("games/catchfood/sprites/screen_game_lost.png", 320, 320)
            )
        }
    }
}

private fun decodeSampledAsset(
    context: Context,
    path: String,
    requestedWidthPx: Float,
    requestedHeightPx: Float
): ImageBitmap? = runCatching {
    val bounds = BitmapFactory.Options().apply { inJustDecodeBounds = true }
    context.assets.open(path).use { BitmapFactory.decodeStream(it, null, bounds) }

    val fitScale = minOf(
        requestedWidthPx / bounds.outWidth,
        requestedHeightPx / bounds.outHeight
    )
    val fittedWidth = bounds.outWidth * fitScale
```

continuacion

```
val fittedHeight = bounds.outHeight * fitScale

var sampleSize = 1
while (
    bounds.outWidth / (sampleSize * 2) >= fittedWidth &&
    bounds.outHeight / (sampleSize * 2) >= fittedHeight
) {
    sampleSize *= 2
}

val options = BitmapFactory.Options().apply {
    inSampleSize = sampleSize
    inPreferredConfig = android.graphics.Bitmap.Config.ARGB_8888
}
context.assets.open(path).use { stream ->
    requireNotNull(BitmapFactory.decodeStream(stream, null, options)).asImageBitmap()
}
}.getOrNull()
```

app/src/main/java/com/example/playland2/feature/catchfood/ui/screen/CatchFoodUiState.kt

```
package com.example.playland2.feature.catchfood.ui.screen

data class CatchFoodUiState(
    val score: Int = 0,
    val poisonHits: Int = 0,
    val missedFood: Int = 0,
    val playerState: String = "eat",
    val isPaused: Boolean = false,
    val isGameOver: Boolean = false
)
```

app/src/main/java/com/example/playland2/feature/catchfood/ui/screen/CatchFoodViewModel.kt

```
package com.example.playland2.feature.catchfood.ui.screen

import android.app.Application
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.setValue
import androidx.lifecycle.AndroidViewModel
import androidx.lifecycle.ViewModelScope
import com.example.playland2.feature.catchfood.data.CatchFoodScoreRepository
import kotlinx.coroutines.launch

class CatchFoodViewModel(application: Application) : AndroidViewModel(application) {
    val gameLogic = CatchFoodGameLogic()

    private val scoreRepository = CatchFoodScoreRepository(application)
    private var scoreSavedForCurrentGame = false

    var uiState by mutableStateOf(gameLogic.toUiState())
    private set

    fun updateGame(deltaSeconds: Float) {
        gameLogic.updateGame(deltaSeconds)
        syncUiState()

        if (gameLogic.isGameOver && !scoreSavedForCurrentGame) {
            scoreSavedForCurrentGame = true
            val finalScore = gameLogic.score
            viewModelScope.launch {
                scoreRepository.saveScore(finalScore)
            }
        }
    }

    fun movePlayer(deltaX: Float) {
        gameLogic.movePlayer(deltaX)
    }

    fun setViewportHeight(heightDp: Float) {
        gameLogic.setViewportHeight(heightDp)
    }

    fun pause() {
        if (!uiState.isGameOver) {
            uiState = uiState.copy(isPaused = true)
        }
    }

    fun resume() {
        uiState = uiState.copy(isPaused = false)
    }

    fun restart() {
        gameLogic.restart()
        scoreSavedForCurrentGame = false
        uiState = gameLogic.toUiState()
    }

    private fun syncUiState() {

```

continuacion

```
        if (
            uiState.score != gameLogic.score ||
            uiState.poisonHits != gameLogic.poisonHits ||
            uiState.missedFood != gameLogic.missedFood ||
            uiState.playerState != gameLogic.playerState ||
            uiState.isGameOver != gameLogic.isGameOver
        ) {
            uiState = gameLogic.toUiState(isPaused = uiState.isPaused)
        }
    }

    private fun CatchFoodGameLogic.toUiState(isPaused: Boolean = false) = CatchFoodUiState(
        score = score,
        poisonHits = poisonHits,
        missedFood = missedFood,
        playerState = playerState,
        isPaused = isPaused,
        isGameOver = isGameOver
    )
}
```

app/src/main/java/com/example/playland2/feature/catchfood/ui/screen/CatchFoodLeaderboard.kt

```
package com.example.playland2.feature.catchfood.ui.screen

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.itemsIndexed
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.produceState
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.Font
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.playland2.R
import com.example.playland2.feature.catchfood.data.CatchFoodScoreRepository

@Composable
fun CatchFoodLeaderboard(onBack: () -> Unit) {
    val context = LocalContext.current
    val scoreRepository = remember(context) { CatchFoodScoreRepository(context) }
    val scores by produceState(initialValue = emptyList(), scoreRepository) {
        value = scoreRepository.getTopScores()
    }
    val gameFont = remember { FontFamily(Font(R.font.fredoka_bold)) }

    Box(
        modifier = Modifier
            .fillMaxSize()
            .background(Color(0xFFFCF5E4))
            .padding(horizontal = 32.dp, vertical = 48.dp)
    ) {
        Column(
            modifier = Modifier.fillMaxSize(),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            Text(
                text = "TOP 10",
                fontFamily = gameFont,
                fontSize = 42.sp,
            )
        }
    }
}
```

continuacion

```

        color = Color(0xFFE96E9D)
    )
    Text(
        text = "Puntaje Maximo",
        fontFamily = gameFont,
        fontSize = 22.sp,
        color = Color(0xFF6B3F2D)
    )

    Spacer(Modifier.height(28.dp))

    if (scores.isEmpty()) {
        Text(
            text = "No hay partidas.",
            fontFamily = gameFont,
            fontSize = 20.sp,
            color = Color.DarkGray
        )
        Spacer(Modifier.weight(1f))
    } else {
        LazyColumn(
            modifier = Modifier.weight(1f),
            verticalArrangement = Arrangement.spacedBy(10.dp)
        ) {
            itemsIndexed(scores) { index, score ->
                Row(
                    modifier = Modifier
                        .fillMaxWidth()
                        .background(
                            color = if (index == 0) Color(0xFFFFE2A8) else
                                Color.White,
                            shape = RoundedCornerShape(18.dp)
                        )
                    .padding(horizontal = 24.dp, vertical = 12.dp),
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text(
                        text = "${index + 1}.",
                        fontFamily = gameFont,
                        fontSize = 22.sp,
                        color = Color(0xFF6B3F2D)
                    )
                    Spacer(Modifier.width(20.dp))
                    Text(
                        text = score.toString(),
                        fontFamily = gameFont,
                        fontWeight = FontWeight.Bold,
                        fontSize = 24.sp,
                        color = Color(0xFF4F8D3A)
                    )
                }
            }
        }
    }

    Button(
        onClick = onBack,
        modifier = Modifier

```

continuacion

```
                .fillMaxWidth()
                .height(58.dp),
                colors = ButtonDefaults.buttonColors(containerColor = Color(0xFFD95B55)),
                shape = RoundedCornerShape(18.dp)
            ) {
                Text("VOLVER", fontFamily = gameFont, fontSize = 20.sp)
            }
        }
    }
}
```

app/src/main/java/com/example/playland2/feature/catchfood/ data/CatchFoodScoreEntity.kt

```
package com.example.playland2.feature.catchfood.data

import androidx.room.Entity
import androidx.room.PrimaryKey

//aqui defino la tabla y las columnas de la table de record
@Entity(tableName = "catch_food_scores")
data class CatchFoodScoreEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0,
    val foodCaught: Int
)
```

app/src/main/java/com/example/playland2/feature/catchfood/data/CatchFoodScoreDao.kt

```
package com.example.playland2.feature.catchfood.data

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.Query

@Dao
interface CatchFoodScoreDao {
    //aca inserto un registro nuevo
    @Insert
    suspend fun insert(score: CatchFoodScoreEntity)

    @Query(
        """
        SELECT foodCaught
        FROM catch_food_scores
        ORDER BY foodCaught DESC, id ASC
        LIMIT 10
        """
    )

    //devuelvo enteros
    suspend fun getTopTenScores(): List<Int>
}
```

app/src/main/java/com/example/playland2/feature/catchfood/data/CatchFoodDatabase.kt

```
package com.example.playland2.feature.catchfood.data

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(
    entities = [CatchFoodScoreEntity::class],
    version = 1,
    exportSchema = false
)
abstract class CatchFoodDatabase : RoomDatabase() {
    abstract fun scoreDao(): CatchFoodScoreDao

    companion object {
        @Volatile
        private var instance: CatchFoodDatabase? = null

        //con esta funcion me traigo la bsd
        fun getInstance(context: Context): CatchFoodDatabase = instance ?:
            synchronized(this) {
                instance ?: Room.databaseBuilder(
                    context.applicationContext,
                    CatchFoodDatabase::class.java,
                    bsd_name
                ).build().also { instance = it }
            }

        private const val bsd_name = "catch_food.db"
    }
}
```

app/src/main/java/com/example/playland2/feature/catchfood/data/CatchFoodScoreRepository.kt

```
package com.example.playland2.feature.catchfood.data

import android.content.Context

class CatchFoodScoreRepository(context: Context) {
    private val scoreDao = CatchFoodDatabase.getInstance(context).scoreDao()

    suspend fun saveScore(score: Int) {
        scoreDao.insert(CatchFoodScoreEntity(foodCaught = score))
    }

    suspend fun getTopScores(): List<Int> = scoreDao.getTopTenScores()
}
```